

Storage Systems in System Design

1. Overview

Storage in system design means choosing where and how application data is stored, retrieved, indexed, cached, replicated, and delivered to users.

In interviews, storage usually covers five major choices:

- **SQL databases** for structured relational data.
- **NoSQL databases** for flexible, scalable, or specialized data models.
- **Key-value stores** for very fast lookup by key.
- **Document databases** for JSON-like application objects.
- **Object/blob storage** for large files such as images, videos, backups, and logs.
- **CDNs** for caching and serving static content close to users.

Definition

A storage system is a component that persists data so that an application can read it later. Different storage systems optimize for different things: consistency, speed, query flexibility, scale, cost, durability, and global delivery.

Why It Matters

Most system design failures come from poor storage choices:

- Using SQL when the workload needs massive write scalability.
- Using NoSQL when strong relational constraints are required.
- Storing large images directly in a database.
- Serving global static assets from one backend server.
- Ignoring indexes, replication, partitioning, or caching.

Where It Is Used in Real Systems

System	Storage Choices
Banking app	SQL database for accounts and transactions
E-commerce app	SQL for orders, document DB for product catalog, object storage for images
URL shortener	Key-value store for short code to long URL mapping
Social media	Document DB or wide-column store for posts, object storage for media, CDN for delivery
Video streaming	Object storage for videos, CDN for global playback
Analytics platform	Columnar/NoSQL storage for high-volume event data

Why Interviewers Ask About It

Interviewers ask storage questions to check whether you can:

- Match data model to access pattern.
- Understand trade-offs between consistency, availability, latency, and scale.
- Avoid common bad designs.
- Explain why one database is better than another for a specific use case.
- Design practical systems like Instagram, YouTube, Dropbox, Twitter, or URL shorteners.

2. Core Idea

The core idea is: **choose storage based on access pattern, not hype.**

Before choosing SQL, NoSQL, object storage, or CDN, ask:

1. What data are we storing?
2. Is the data structured or flexible?
3. How will we query it?
4. How large will it become?
5. Do we need transactions?
6. Do we need strong consistency?
7. Is the data frequently read, frequently written, or both?
8. Is it small records or large files?
9. Is it served globally?

Intuition

Think of storage like different types of storage in a house:

Real-World Storage	System Design Equivalent
Filing cabinet with labeled folders	SQL database
Flexible notebook pages	Document database
Locker with numbered keys	Key-value store
Warehouse for large boxes	Object/blob storage
Local store branch near customers	CDN

You would not store a sofa inside a filing cabinet. Similarly, you should not store a large video file inside a relational database row.

Small Example

Suppose you are designing an e-commerce app.

You need to store:

Data	Best Storage
Users	SQL database
Orders and payments	SQL database
Product catalog	SQL or document DB
Shopping cart sessions	Key-value store
Product images	Object storage
Image delivery to users	CDN
Search index	Search engine such as Elasticsearch/OpenSearch

Step-by-Step Storage Selection

1. **Identify entities** Users, orders, products, images, sessions, logs.
2. **Identify relationships** A user has many orders. An order has many order items. These relationships often fit SQL.
3. **Identify query patterns** Example: "Find all orders for a user" or "Fetch product by ID".
4. **Identify scale** Millions of product image reads may require CDN. Billions of events may require NoSQL or analytics storage.
5. **Identify consistency needs** Payments need strong consistency. Feed likes may tolerate eventual consistency.
6. **Pick storage per data type** Most real systems use multiple storage systems together.

3. Important Subtopics

3.1 SQL Databases

What It Means

SQL databases store data in tables with rows and columns. They use schemas, relationships, indexes, and SQL queries.

Examples:

- PostgreSQL
- MySQL
- Oracle
- SQL Server

Why It Matters

SQL is excellent when data is structured and relationships are important.

Use SQL when you need:

- ACID transactions.
- Joins.
- Strong consistency.
- Constraints such as foreign keys and unique keys.
- Complex queries.
- Financial or business-critical correctness.

Example

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(255) UNIQUE  
);  
  
CREATE TABLE orders (  
  id BIGINT PRIMARY KEY,  
  user_id BIGINT REFERENCES users(id),  
  amount DECIMAL(10, 2),  
  status VARCHAR(20)  
);
```

This is good for e-commerce orders because order data is structured and correctness matters.

Common Interview Angle

Interviewers often ask:

- Why use SQL for payments?
- How do indexes improve query speed?
- How do you scale SQL?
- What are ACID transactions?
- When would SQL become a bottleneck?

3.2 NoSQL Databases

What It Means

NoSQL databases are non-relational databases designed for flexible schemas, high scalability, high write throughput, or specialized access patterns.

Types of NoSQL:

Type	Data Model	Examples
Key-value	Key maps to value	Redis, DynamoDB
Document	JSON-like documents	MongoDB, Couchbase

Type	Data Model	Examples
Wide-column	Rows with dynamic columns	Cassandra, HBase
Graph	Nodes and edges	Neo4j, JanusGraph

Why It Matters

NoSQL is useful when:

- Data does not fit cleanly into tables.
- Schema changes frequently.
- System needs horizontal scalability.
- Workload is very high write volume.
- Queries are simple and predictable.
- Eventual consistency is acceptable.

Example

A social media post stored as a document:

```
{
  "post_id": "p123",
  "user_id": "u456",
  "text": "Hello world",
  "media": ["image1.jpg"],
  "likes": 120,
  "comments": [
    {"user_id": "u789", "text": "Nice post"}
  ]
}
```

This is easier to store as a document than splitting everything into many relational tables.

Common Interview Angle

Interviewers ask:

- Why not use SQL for everything?
- What is eventual consistency?
- How does NoSQL scale horizontally?
- What are the downsides of NoSQL?
- Which NoSQL type fits this use case?

3.3 Key-Value Store

What It Means

A key-value store stores data as:

```
key -> value
```

You retrieve the value only if you know the key.

Examples:

- Redis
- Memcached
- Amazon DynamoDB
- Riak

Why It Matters

Key-value stores are extremely fast for simple lookups. They are commonly used for caching, sessions, counters, rate limiting, and metadata lookup.

Example

URL shortener:

```
abc123 -> https://example.com/very/long/url
```

When a user visits `short.ly/abc123`, the backend looks up `abc123` and redirects to the long URL.

Common Interview Angle

Interviewers ask:

- Why is a key-value store good for a URL shortener?
- What happens if the key does not exist?
- Should the value be stored forever or expire?
- How do you handle hot keys?
- How do you shard keys?

3.4 Document Database

What It Means

A document database stores semi-structured documents, usually JSON or BSON.

Examples:

- MongoDB
- CouchDB
- Firestore
- Couchbase

Why It Matters

Document DBs are useful when application data naturally looks like nested objects and schema may evolve over time.

Example

Product catalog:

```
{
  "product_id": "p100",
  "name": "Laptop",
  "category": "Electronics",
  "price": 75000,
  "attributes": {
    "ram": "16GB",
    "storage": "512GB SSD",
    "processor": "i7"
  }
}
```

Different product categories can have different attributes without requiring many nullable SQL columns.

Common Interview Angle

Interviewers ask:

- Why use MongoDB for product catalogs?
- What are embedded documents vs references?
- What happens when documents grow too large?
- How do indexes work in a document DB?
- When is document DB worse than SQL?

3.5 Object / Blob Storage

What It Means

Object storage stores large unstructured data as objects. Each object usually has:

- Binary content.
- Metadata.
- Unique object key.
- Bucket/container.

Examples:

- Amazon S3
- Google Cloud Storage
- Azure Blob Storage

- MinIO

Why It Matters

Object storage is designed for massive scale, high durability, and low cost for large files.

Use it for:

- Images.
- Videos.
- PDFs.
- Backups.
- Logs.
- Machine learning datasets.
- Static website assets.

Example

Instagram-style image upload:

```
graph TD
    A[User uploads image] --> B[Backend validates upload]
    B --> C[Image stored in object storage]
    C --> D[Metadata stored in database]
    D --> E[Image delivered through CDN]
```

The database stores only metadata:

```
image_id, user_id, object_url, upload_time, caption
```

The actual image bytes live in object storage.

Common Interview Angle

Interviewers ask:

- Why not store images in SQL?
- What does object storage provide?
- How do you generate object keys?

- How do you handle private files?
- How do you serve files efficiently worldwide?

3.6 CDN

What It Means

A CDN, or Content Delivery Network, is a distributed network of servers that cache content near users.

Examples:

- Cloudflare
- Akamai
- Amazon CloudFront
- Fastly
- Google Cloud CDN

Why It Matters

CDNs reduce:

- Latency.
- Backend load.
- Bandwidth cost.
- Origin server traffic.

They improve:

- Global performance.
- Availability.
- Static content delivery.
- Video and image loading.

Example

Without CDN:

```
User in India -> US server -> Image response
```

With CDN:

```
User in India -> Nearby CDN edge in Mumbai -> Image response
```

Common Interview Angle

Interviewers ask:

- How does CDN caching work?
- What is cache invalidation?
- What happens on a cache miss?
- What content should be cached?
- How do CDNs help scalability?

3.7 Indexing

What It Means

An index is a data structure that speeds up reads by avoiding full scans.

Why It Matters

Without an index, a database may scan every row or document.

Example

```
CREATE INDEX idx_orders_user_id ON orders(user_id);
```

Now this query becomes faster:

```
SELECT * FROM orders WHERE user_id = 101;
```

Common Interview Angle

Interviewers expect you to mention that indexes improve reads but add overhead to writes and storage.

3.8 Replication

What It Means

Replication means keeping copies of data on multiple machines.

Why It Matters

Replication improves:

- Availability.
- Read scalability.
- Fault tolerance.
- Disaster recovery.

Example

```
Primary DB -> Replica 1  
           -> Replica 2  
           -> Replica 3
```

Writes go to primary. Reads may go to replicas.

Common Interview Angle

Interviewers ask about replica lag, failover, read replicas, and consistency.

3.9 Sharding / Partitioning

What It Means

Sharding splits data across multiple machines.

Why It Matters

A single database machine has limits. Sharding helps scale storage and throughput.

Example

```
Users with id 1-1M      -> Shard 1  
Users with id 1M-2M    -> Shard 2  
Users with id 2M-3M    -> Shard 3
```

Or hash-based:

```
shard = hash(user_id) % number_of_shards
```

Common Interview Angle

Interviewers ask:

- Which shard key would you choose?
- What is a hot shard?
- How do you rebalance shards?
- Why is cross-shard transaction hard?

4. Real-World Example

Example: Designing Storage for Instagram

Instagram-like systems need to store users, posts, comments, likes, images, feeds, and notifications.

Data	Storage Choice	Reason
User profile	SQL or document DB	Structured profile data
Login credentials	SQL	Strong consistency and constraints
Follow relationships	Graph DB, SQL, or wide-column store	Relationship-heavy access
Image metadata	SQL or document DB	Query by user, post, timestamp
Image files	Object storage	Large binary files
Feed cache	Key-value store	Fast feed retrieval
Likes count	Key-value store or counter service	High write volume
Comments	SQL, document DB, or wide-column store	Depends on scale and query pattern
Static media delivery	CDN	Low-latency global delivery

Upload Flow

1. User uploads image from app.
2. Backend authenticates user.
3. Backend stores image in object storage.
4. Backend writes metadata to database.
5. CDN caches image when users request it.
6. Feed service updates followers' feeds or computes feed on read.

Read Flow

```
User opens feed
  |
  v
Feed service fetches post IDs from cache
  |
  v
Post metadata fetched from DB
  |
  v
Image URLs returned to client
  |
  v
Client loads images from CDN
```

5. Diagrams / Mental Models

Storage Selection Mental Model

What are you storing?

```

|
+-- Structured relational data?
|   |
|   +-- Need joins/transactions? -> SQL
|
+-- JSON-like flexible objects?
|   |
|   +-- Need nested fields? -> Document DB
|
+-- Simple lookup by key?
|   |
|   +-- Need very fast access? -> Key-value store
|
+-- Large binary files?
|   |
|   +-- Images/videos/PDFs? -> Object storage
|
+-- Static content for global users?
|   |
|   +-- Cache near users -> CDN

```

Read Path with CDN

```

Client
|
v
CDN Edge
|
+-- Cache hit -> Return content immediately
|
+-- Cache miss -> Fetch from origin/object storage
                        |
                        v
                    Cache content
                        |
                        v
                    Return to client

```

Database Scaling Mental Model

Start:
Single SQL database

When reads increase:
Add indexes + read replicas + cache

When writes increase:

Optimize schema + batch writes + partition data

When one machine is not enough:
Shard data

When data model changes:
Consider NoSQL or specialized storage

Storage Responsibility Map

Component	Responsibility
SQL DB	Correct relational records
NoSQL DB	Scalable flexible data
Key-value store	Fast key lookup/cache/session
Document DB	JSON-like application objects
Object storage	Large binary files
CDN	Global cached delivery
Search index	Text search and ranking
Data warehouse	Analytics and reporting

6. Common Interview Questions

1. What is the difference between SQL and NoSQL?

Answer: SQL databases store structured data in tables and support relationships, joins, schemas, and ACID transactions. NoSQL databases use non-relational models such as key-value, document, wide-column, or graph and are often chosen for scalability, flexible schema, or high throughput.

Key points expected:

- SQL: structured schema, joins, ACID.
- NoSQL: flexible schema, horizontal scaling, specialized data models.
- Choice depends on use case.

Common mistakes:

- Saying NoSQL means "no SQL at all".
- Saying NoSQL is always faster.
- Saying SQL cannot scale.

2. When would you choose SQL?

Answer: Choose SQL when data is structured, relationships matter, strong consistency is required, and transactions are important. Examples include banking, orders, inventory, payments, and user accounts.

Key points expected:

- ACID transactions.
- Data integrity.
- Joins and constraints.
- Mature query support.

Common mistakes:

- Ignoring SQL for large systems.
- Forgetting indexing and replication.

3. When would you choose NoSQL?

Answer: Choose NoSQL when the data model is flexible, write volume is very high, horizontal scaling is important, or the query pattern is simple and predictable. Examples include event logs, feeds, IoT data, and large-scale user activity.

Key points expected:

- Flexible schema.
- Horizontal scalability.
- Eventual consistency may be acceptable.
- Pick the correct NoSQL type.

Common mistakes:

- Treating all NoSQL databases as the same.
- Ignoring consistency requirements.

4. Why are key-value stores fast?

Answer: Key-value stores are fast because they use a simple access pattern: lookup by key. Many are memory-based or optimized for hash-based access, so they avoid complex joins and query planning.

Key points expected:

- Direct key lookup.
- Simple data model.
- Often in-memory.
- Great for cache and sessions.

Common mistakes:

- Assuming key-value stores are good for complex queries.
- Forgetting eviction and expiration policies.

5. Why should large files not usually be stored directly in SQL databases?

Answer: Large files increase database size, slow backups, make replication heavy, and put unnecessary load on a system optimized for structured records. It is better to store files in object storage and keep metadata in the database.

Key points expected:

- Object storage is cheaper and more scalable for files.
- DB should store metadata and object URL/key.
- CDN can serve files efficiently.

Common mistakes:

- Saying it is impossible to store files in SQL.
- Forgetting that small blobs may sometimes be acceptable.

6. What is object storage?

Answer: Object storage stores unstructured data such as images, videos, PDFs, backups, and logs as objects inside buckets. Each object has data, metadata, and a unique key.

Key points expected:

- Good for large binary data.
- Durable and scalable.
- Accessed using object keys or URLs.
- Often used with CDN.

Common mistakes:

- Confusing object storage with block storage.
- Treating it like a relational database.

7. What is a CDN and why is it used?

Answer: A CDN is a distributed network of edge servers that caches content close to users. It reduces latency, backend load, and bandwidth usage.

Key points expected:

- Edge caching.
- Cache hit and cache miss.
- Origin server.
- Global performance.

Common mistakes:

- Saying CDN stores all application data.
- Ignoring cache invalidation.

8. What happens during a CDN cache miss?

Answer: If the CDN edge does not have the requested content, it fetches it from the origin server or object storage, returns it to the user, and usually caches it for future requests.

Key points expected:

- Client asks CDN.

- CDN checks cache.
- CDN fetches from origin on miss.
- Response may be cached based on TTL/cache headers.

Common mistakes:

- Assuming cache miss is an error.
- Forgetting origin load during cache misses.

9. What is eventual consistency?

Answer: Eventual consistency means that after a write, all replicas will eventually converge to the same value, but reads immediately after the write may see stale data.

Key points expected:

- Temporary inconsistency.
- Useful for availability and scale.
- Common in distributed NoSQL systems.
- Not suitable for every use case.

Common mistakes:

- Thinking eventual consistency means data loss.
- Using it for banking balances without careful design.

10. How do you decide the database for a URL shortener?

Answer: The main access pattern is short code to long URL lookup. A key-value store is a natural fit. A SQL database can also work at smaller scale, especially if analytics, ownership, and constraints are needed.

Key points expected:

- Key-value lookup.
- Low latency redirect.
- Handle expiration.
- Cache popular URLs.
- Persist mappings durably.

Common mistakes:

- Over-designing with complex joins.
- Ignoring collision handling for short codes.

11. How do indexes affect database performance?

Answer: Indexes speed up reads by allowing the database to find matching rows faster, but they slow down writes because indexes must be updated whenever data changes. They also consume extra storage.

Key points expected:

- Faster reads.

- Slower writes.
- Extra storage.
- Choose indexes based on query patterns.

Common mistakes:

- Adding indexes to every column.
- Forgetting composite index order.

12. How would you store user sessions?

Answer: User sessions are commonly stored in a key-value store like Redis because sessions are accessed by session ID, need low latency, and often have expiration.

Key points expected:

- session_id -> session data.
- TTL expiration.
- Fast lookup.
- Shared session store across backend servers.

Common mistakes:

- Storing sessions only in local server memory in a horizontally scaled system.
- Forgetting logout/session invalidation.

7. Deep-Dive Questions

1. How do you choose between embedding and referencing in a document database?

Answer: Embed data when it is small, tightly related, and usually read together. Reference data when it is large, shared by many documents, updated frequently, or can grow without bound.

Example:

- Embed shipping address inside an order snapshot.
- Reference user profile from many posts.

Interviewers expect you to discuss read patterns, update frequency, document growth, and duplication.

2. How do you handle hot keys in a key-value store?

Answer: A hot key is a key receiving too much traffic, causing one node to become overloaded. Solutions include caching at multiple layers, key splitting, request coalescing, read replicas, local caching, and sometimes redesigning the data model.

Example:

```
celebrity_post_likes -> too many writes
```

Instead of one counter, use multiple counter shards:

```
celebrity_post_likes_0
celebrity_post_likes_1
...
celebrity_post_likes_99
```

Then sum them when needed.

3. How does sharding affect transactions?

Answer: Sharding makes transactions harder because related data may live on different machines. A transaction across shards requires distributed coordination, which increases latency and complexity.

Interview answer should mention:

- Single-shard transaction is simpler.
- Cross-shard transaction is expensive.
- Good shard key design can keep related data together.
- Some systems use eventual consistency or sagas.

4. How do you invalidate CDN cache after updating an image or file?

Answer: Common approaches are TTL-based expiry, explicit purge/invalidation, or versioned URLs.

Best practical method:

```
/profile/u123/v1.jpg
/profile/u123/v2.jpg
```

When the image changes, use a new URL. This avoids waiting for old CDN caches to expire.

5. How would you design storage for a video streaming platform?

Answer: Store video files in object storage, store metadata in SQL or document DB, use a CDN for global delivery, keep user watch history in SQL/NoSQL, store recommendations in a specialized service, and use analytics storage for events.

Expected points:

- Videos are large blobs, so use object storage.
- CDN is mandatory for low-latency playback.
- Metadata is separate from video bytes.
- Transcoded versions may be stored as separate objects.
- User history and likes need database storage.

8. Comparison Tables

SQL vs NoSQL

Feature	SQL	NoSQL
Data model	Tables, rows, columns	Key-value, document, wide-column, graph
Schema	Fixed/predefined	Flexible or dynamic
Query language	SQL	Varies by database
Joins	Strong support	Limited or avoided
Transactions	Strong ACID support	Varies, often limited or scoped
Scaling	Traditionally vertical, can scale horizontally with effort	Designed for horizontal scaling
Best for	Structured relational data	Flexible or high-scale workloads
Examples	PostgreSQL, MySQL	MongoDB, Cassandra, DynamoDB, Redis
Interview use case	Payments, orders, inventory	Feeds, logs, sessions, large-scale events

Key-Value Store vs Document Database

Feature	Key-Value Store	Document Database
Data access	By key only	Query by fields and indexes
Value structure	Opaque or simple	JSON/BSON-like document
Query flexibility	Low	Medium to high
Speed	Very high for key lookup	Good, depends on indexes
Best for	Cache, sessions, counters	Product catalog, profiles, posts
Examples	Redis, DynamoDB, Memcached	MongoDB, Couchbase, Firestore

Object Storage vs Database

Feature	Object Storage	Database
Stores	Files/blobs	Structured or semi-structured records
Query ability	Usually by object key	Rich queries/indexes
Cost for large files	Low	Higher
Durability	Very high	Depends on DB setup
Access pattern	Upload/download object	Query/update records
Best for	Images, videos, backups	Users, orders, metadata

CDN vs Cache

Feature	CDN	Application Cache
Location	Globally distributed edge servers	Near backend/application
Purpose	Deliver static/media content globally	Reduce backend/database load
Example	CloudFront, Cloudflare	Redis, Memcached
Cached data	Images, JS, CSS, videos, files	Query results, sessions, computed data
User-facing	Yes, often directly serves users	Usually internal

Replication vs Sharding

Feature	Replication	Sharding
Meaning	Copy same data to multiple nodes	Split data across nodes
Main goal	Availability and read scaling	Write/storage scaling
Data on each node	Same or mostly same	Different subset
Failure handling	Replica can take over	Only affected shard is impacted
Complexity	Moderate	Higher
Common issue	Replica lag	Hot shards and rebalancing

Block Storage vs Object Storage vs File Storage

Feature	Block Storage	File Storage	Object Storage
Unit	Blocks	Files/directories	Objects
Access	Attached like disk	File path	Object key/API
Best for	Databases, VMs	Shared file systems	Large media/files/backups
Examples	EBS, persistent disks	NFS, EFS	S3, GCS, Azure Blob
Scale	Good but attached	Shared filesystem limits	Massive scale

9. Common Mistakes

- Choosing NoSQL just because the system is large.
- Choosing SQL for large media files instead of object storage.
- Saying SQL cannot scale.
- Saying NoSQL has no consistency.
- Ignoring the actual query pattern.
- Forgetting that most real systems use multiple storage types.
- Adding indexes without considering write overhead.
- Storing user sessions in one backend server's memory.
- Forgetting cache invalidation and TTLs.

- Treating CDN as a database.
- Ignoring data durability requirements.
- Choosing a shard key that creates hot shards.
- Forgetting backup and disaster recovery.
- Confusing object storage with document storage.
- Assuming eventual consistency is acceptable for every feature.

10. Edge Cases / Special Cases

1. SQL Can Store JSON

Modern SQL databases like PostgreSQL support JSON columns. This can be useful when most data is relational but a few fields are flexible.

Interview trap:

Do not claim that flexible data always requires NoSQL.

2. NoSQL Can Support Transactions

Some NoSQL systems support transactions, but often with limitations or different performance characteristics.

Interview trap:

Do not say "NoSQL never has transactions."

3. Small Files May Be Stored in DB Sometimes

For tiny files or tightly coupled binary data, storing blobs in a database may be acceptable. But at scale, object storage is usually better.

4. CDN Can Serve Dynamic Content Too

CDNs are commonly used for static files, but modern CDNs can also cache API responses or run edge logic. However, dynamic caching must be designed carefully.

5. Cache Invalidation Is Hard

If data changes but CDN/cache still serves old data, users see stale content. Use TTLs, explicit invalidation, or versioned URLs.

6. Eventual Consistency Can Affect UX

Example:

```
User updates profile picture.  
Some users see old picture for a few seconds.
```

This may be acceptable. But stale account balance is usually not acceptable.

7. Hot Partitions Can Break Scalable Systems

Even if a database is sharded, bad key design can overload one shard.

Bad shard key:

```
date = today
```

If all writes go to today's partition, one shard becomes hot.

8. Metadata and Data Are Often Stored Separately

For media systems:

- Object storage stores bytes.
- Database stores metadata.
- CDN serves content.

This separation is a common interview point.

11. How to Explain in Interview

"Storage choice depends on the data model and access pattern. For structured relational data like users, orders, and payments, I would use SQL because it gives transactions, constraints, and joins. For flexible or high-scale data, I may use NoSQL, choosing the specific type based on access pattern: key-value for fast lookups and caching, document DB for JSON-like objects, and wide-column stores for massive write-heavy workloads. For large files like images and videos, I would store the bytes in object storage and keep metadata in a database. To serve static content globally with low latency, I would put a CDN in front of object storage or the origin server."

12. Quick Revision Notes

Key Definitions

- **SQL:** Relational database with tables, schemas, joins, and ACID transactions.
- **NoSQL:** Non-relational database family optimized for flexible models or scale.
- **Key-value store:** Stores key to value mappings for fast lookup.
- **Document DB:** Stores JSON-like documents with flexible structure.
- **Object/blob storage:** Stores large unstructured files as objects.
- **CDN:** Globally distributed cache for serving content near users.
- **Index:** Data structure that speeds up reads.
- **Replication:** Copying data to multiple nodes.
- **Sharding:** Splitting data across multiple nodes.

Important Points

- Pick storage based on access pattern.
- SQL is best for relationships and correctness.

- NoSQL is not one thing; identify the type.
- Key-value stores are great for cache, sessions, and direct lookup.
- Document DBs are good for nested, flexible objects.
- Object storage is best for large files.
- CDN improves global content delivery.
- Indexes speed reads but slow writes.
- Replication improves availability.
- Sharding improves scale but adds complexity.

Common Comparisons

Comparison	Main Difference
SQL vs NoSQL	Relational correctness vs flexible/scalable data models
Key-value vs Document DB	Lookup by key vs queryable JSON-like documents
Object storage vs DB	Large files vs queryable records
CDN vs Cache	Global edge delivery vs internal speedup
Replication vs Sharding	Copy data vs split data

Must-Remember Facts

- Do not store large videos directly in SQL for a scalable system.
- Store file metadata in DB and bytes in object storage.
- CDN sits close to users and caches content.
- Cache miss goes to origin.
- SQL is still widely used in large systems.
- NoSQL may sacrifice some consistency for availability and scale.
- Shard key choice is critical.
- Most production systems are polyglot persistence systems.

Interview Traps

- "NoSQL is always faster."
- "SQL cannot scale."
- "CDN is only for images."
- "Object storage is the same as a database."
- "Eventual consistency means wrong data forever."
- "Add indexes everywhere."
- "Use one database for everything."

13. Practice Tasks

Task 1: Pick Storage for an E-Commerce System

For each item, choose SQL, document DB, key-value store, object storage, or CDN:

Item	Your Choice
User accounts	
Orders	
Product images	
Product catalog	
Shopping cart	
Session tokens	
Invoice PDFs	
Static website assets	

Expected direction:

- Users/orders: SQL.
- Images/PDFs: object storage.
- Static assets/images: CDN.
- Cart/session: key-value store.
- Catalog: SQL or document DB depending on attributes.

Task 2: Design a URL Shortener Storage Layer

Write the schema or key-value structure for:

```
short_code -> long_url
```

Think about:

- Expiry time.
- User ownership.
- Analytics.
- Collision handling.
- Cache for popular URLs.

Task 3: Draw Instagram Storage Architecture

Draw components for:

- User metadata.
- Post metadata.
- Image storage.
- Feed cache.
- CDN.

Then explain one read path and one write path.

Task 4: SQL Index Exercise

Given this query:

```
SELECT * FROM orders
WHERE user_id = 10
ORDER BY created_at DESC;
```

Suggest a useful index.

Expected:

```
CREATE INDEX idx_orders_user_created
ON orders(user_id, created_at DESC);
```

Task 5: Cache Miss Simulation

Trace what happens:

```
User requests /image/p1.jpg
CDN does not have it
Origin has it
```

Expected flow:

```
Client -> CDN -> Origin/Object Storage -> CDN caches -> Client
```

Task 6: Compare Two Designs

Design A:

```
Store image bytes in SQL database.
```

Design B:

```
Store image in object storage and image metadata in SQL.
```

Explain why Design B is usually better at scale.

Task 7: Implement a Tiny Key-Value Store in Python

```
class KeyValueStore:
    def __init__(self):
        self.data = {}

    def put(self, key, value):
        self.data[key] = value

    def get(self, key):
        return self.data.get(key)

    def delete(self, key):
        self.data.pop(key, None)

store = KeyValueStore()
store.put("abc123", "https://example.com")
print(store.get("abc123"))
store.delete("abc123")
print(store.get("abc123"))
```

Extend it with:

- TTL support.
- Hit/miss counter.
- Maximum size eviction.

Task 8: Explain Storage for YouTube

Try explaining:

- Where video files are stored.
- Where video metadata is stored.
- How video is delivered globally.
- Where comments and likes are stored.
- Where watch history is stored.

14. Final Cheat Sheet

Core Definition

Storage systems persist application data. In system design, the main skill is choosing the right storage type for the data model, access pattern, consistency need, and scale.

Why It Matters

Wrong storage choices cause slow queries, high cost, poor scalability, data inconsistency, and operational complexity.

Most Asked Questions

- SQL vs NoSQL?
- When to use SQL?
- When to use NoSQL?
- What is a key-value store?
- What is a document DB?
- Why store files in object storage?
- What is a CDN?
- What is cache invalidation?
- What is eventual consistency?
- How do replication and sharding differ?

Common Comparisons

Topic	One-Line Difference
SQL vs NoSQL	SQL is relational and transaction-friendly; NoSQL is flexible and scale-oriented.
Key-value vs Document DB	Key-value gives direct lookup; document DB supports structured documents and field queries.
Object storage vs SQL	Object storage stores large files; SQL stores structured records and metadata.
CDN vs Object Storage	Object storage persists files; CDN caches and delivers them near users.
Replication vs Sharding	Replication copies data; sharding splits data.

One-Line Interview Answer

"I choose storage based on access pattern: SQL for relational transactional data, NoSQL for flexible or high-scale data, key-value stores for fast lookups and caching, document DBs for JSON-like objects, object storage for large files, and CDNs for low-latency global content delivery."